

Code Injection with Token Sequences

Document #: P3294R0
Date: 2024-05-22
Project: Programming Language C++
Audience: SG7, EWG
Reply-to: Andrei Alexandrescu, NVIDIA
<andrei@nvidia.com>
Barry Revzin
<barry.revzin@gmail.com>
Daveed Vandevoorde
<daveed@edg.com>

Contents

- [1 Introduction](#)
- [2 A Comparison of Injection Models](#)
 - [2.1 The Spec API](#)
 - [2.2 The CodeReckons API](#)
 - [2.3 String Injection](#)
 - [2.4 Fragments](#)
- [3 Token Sequences](#)
 - [3.1 @tokens literal](#)
 - [3.2 Quoting into a token sequence](#)
 - [3.3 Examples](#)
 - [3.4 Logging Vector: Cloning a Type](#)
 - [3.5 Logging Vector II: Cloning with Modifications](#)
- [4 Hygienic Macros](#)
 - [4.1 Forwarding](#)
 - [4.2 Assertion](#)
 - [4.3 String Interpolation](#)
 - [4.4 Control Flow Operator](#)
 - [4.5 Alternate Syntax](#)
- [5 Proposal](#)
- [6 References](#)

§ 1 Introduction

This paper is proposing augmenting [[P2996R3](#)] to add code injection in the form of token sequences.

We consider the motivation for this feature to some degree pretty obvious, so we will not repeat it here, since there are plenty of other things to cover here. Instead we encourage readers to read some other

papers on the topic ([[P0707R4](#)], [[P0712R0](#)], [[P1717R0](#)], [[P2237R0](#)]).

§ 2 A Comparison of Injection Models

There are a lot of things that make code injection in C++ difficult, and the most important problem to solve first is: what will the actual injection mechanism look like? Not its syntax specifically, but what is the shape of the API that we want to expose to users? We hope in this section to do a thorough job of comparing the various semantic models we're aware of to help explain why we came to the conclusion that we came to.

If you're not interested in this journey, you can simply skip to the [next section](#).

Here, we will look at a few interesting examples for injection and how different models can implement them. The examples aren't necessarily intended to be the most compelling examples that exist in the wild. Instead, they're hopefully representative enough to cover a wide class of problems. They are:

1. Implementing the storage for `std::tuple<Ts...>`
2. Implementing `std::enable_if` without resorting to class template specialization
3. Implementing properties (i.e. given a name like `"author"` and a type like `std::string`, emit a member `std::string m_author`, a getter `get_author()` which returns a `std::string const&` to that member, and a setter `set_author()` which takes a new value of type `std::string const&` and assigns the member).
4. Implement postfix increment in terms of prefix increment.

§ 2.1 The Spec API

In P2996, the injection API is based on a function `define_class()` which takes a range of spec objects. But `define_class()` is a really clunky API, because invoking it is an expression - but we want to do it in contexts that want a declaration. So a simple example of injecting a single member `int` named `x` is:

```
struct C;
static_assert(is_type(define_class(^C,
    {data_member_spec{.name="x", .type=^int}})));
```

We are already separately proposing `consteval` blocks [[P3289R0](#)] and we would like to inject each spec more directly, without having to complete `C` in one go. As in:

```
struct C {
    consteval {
        inject(data_member_spec{.name="x", .type=^int});
    }
};
```

Here, `std::meta::inject` is a metafunction that takes a spec, which gets injected into the context begin by the `consteval` block that our evaluation started in as a side-effect.

We already think of this as an improvement. But let's go through several use-cases to expand the API and see how well it holds up.

§ 2.1.1 `std::tuple`

The tuple use-case was already supported by P2996 directly with `define_class()` (even as we think it'd be better as a member pack), but it's worth just showing what it looks like with a direct injection API instead:

```
template <class... Ts>
struct Tuple {
    consteval {
        std::array types{^Ts...};
        for (size_t i = 0; i != types.size() ;++i) {
            inject(data_member_spec{.name=std::format("_{}", i),
                                     .type=types[i]});
        }
    }
};
```

§ 2.1.2 `std::enable_if`

Now, `std::enable_if` has already been obsolete technology since C++20. So implementing it, specifically, is not entirely a motivating example. However, the general idea of `std::enable_if` as *conditionally* having a member type is a problem that has no good solution in C++ today.

The spec API along with injection does allow for a clean solution here. We would just need to add an `alias_spec` construct to get the job done:

```
template <bool B, class T=void>
struct enable_if {
    consteval {
        if (B) {
            inject(alias_spec{.name="type", .type=^T});
        }
    }
};
```

So far so good.

§ 2.1.3 Properties

Now is when the spec API really goes off the rails. We've shown data members and extended it to member aliases. But how do we support member functions?

We want to be able to add a property with a given name and type that adds a member of that type and a getter and setter for it. For instance, we want this code:

```
struct Book {
    constexpr {
        property("author", ^std::string);
        property("title", ^std::string);
    }
};
```

to emit a class with two members (`m_author` and `m_title`), two getters that each return a `std::string const&` (`get_author()` and `get_title()`) and two setters that each take a `std::string const&` (`set_author()` and `set_title()`). Fairly basic property.

We start by injecting the member:

```
constexpr auto property(string_view name, meta::info type)
    -> void
{
    inject(data_member_spec{.name=std::format("m_{}", name),
                           .type=type});

    // ...
}
```

Now, we need to inject two functions. We'll need a new kind of spec for that case, and then we can use a lambda for the function body. Let's start with the getter:

```
constexpr auto property(string_view name, meta::info type)
    -> void
{
    inject(data_member_spec{.name=std::format("m_{}", name),
                           .type=type});

    inject(function_member_spec{
        .name=std::format("get_{}", name),
        .body=^[](auto const& self) -> auto const& {
            return self./* ??? */;
        }
    });

    // ...
}
```

Okay. Uh. What do we return? For the title property, this needs to be `return self.m_title;`, but how do we spell that? We just... can't. We have our member right there (the `data_member_spec` we're injecting), so you might think we could try to capture it:

```

constexpr auto property(string_view name, meta::info type)
    -> void
{
    auto member = inject(data_member_spec{
        .name=std::format("m_{}", name),
        .type=type
    });

    inject(function_member_spec{
        .name=std::format("get_{}", name),
        .body=[member](auto const& self) -> auto const& {
            return self.[:member:];
        }
    });

    // ...
}

```

But that doesn't work - in order to splice member, it needs to be a constant expression - and it's not in this context.

Now, the body of the lambda isn't going to be evaluated in this constant evaluation, so it's possible to maybe come up with some mechanism to pass a context through - such that from the body we *can* simply splice member. We basically need to come up with a way to defer this instantiation.

For now, let's try a spelling like this:

```

constexpr auto property(string_view name, meta::info type)
    -> void
{
    auto member = inject(data_member_spec{
        .name=std::format("m_{}", name),
        .type=type
    });

    inject(function_member_spec{
        .name=std::format("get_{}", name),
        .body=defer(member, ^[]<std::meta::info M>(auto const& self) ->
            auto const& {
                return self.[:M:];
            })
    });

    // ...
}

```

and we can do something similar with the setter:

```

constexpr auto property(string_view name, meta::info type)
    -> void

```

```

{
    auto member = inject(data_member_spec{
        .name=std::format("m_{}", name),
        .type=type
    });

    inject(function_member_spec{
        .name=std::format("get_{}", name),
        .body=defer(member, ^[]<std::meta::info M>(auto const& self) ->
            auto const& {
                return self.[:M:];
            })
    });

    inject(function_member_spec{
        .name=std::format("set_{}", name),
        .body=defer(member, ^[]<std::meta::info M>(auto& self, typename
            [:type_of(M):] const& x) -> void {
                self.[:M:] = x;
            })
    });
}

```

Now we run into the next problem: what actual signature is the compiler going to inject for `get_author()` and `set_author()`? First, we're introducing this extra non-type template parameter which we have to know to strip off somehow. Secondly, we're always taking the object parameter as a deduced parameter. How does the API know what we mean by that?

```

struct Book {
    // do we mean this
    auto get_author(this Book const& self) -> auto const& { return
        self.m_author; }
    auto set_author(this Book& self, string const& x) -> void {
        self.m_author = x; }

    // or this
    auto get_author(this auto const& self) -> auto const& { return
        self.m_author; }
    auto set_author(this auto& self, string const& x) -> void {
        self.m_author = x; }
};

```

That is: how does the compiler know whether we're injecting a member function or a member function template? Our lambda has to be generic either way. Moreover, even if we actually wanted to inject a function template, it's possible that we might want some parameter to be dependent but *not* the object parameter.

Well, we could provide another piece of information to `function_member_spec`: the signature directly:

```

template <class T> using getter_type = auto() const -> T const&;
template <class T> using setter_type = auto(T const&) -> void;

```

```

constexpr auto property(string_view name, meta::info type)
    -> void
{
    auto member = inject(data_member_spec{
        .name=std::format("m_{}", name),
        .type=type
    });

    inject(function_member_spec{
        .name=std::format("get_{}", name),
        .signature=substitute(^getter_type, {^type}),
        .body=defer(member, ^[]<std::meta::info M>(auto const& self) ->
            auto const& {
                return self.[:M:];
            })
    });

    inject(function_member_spec{
        .name=std::format("set_{}", name),
        .signature=substitute(^setter_type, {^type}),
        .body=defer(member, ^[]<std::meta::info M>(auto& self, typename
            [:type_of(M):] const& x) -> void {
                self.[:M:] = x;
            })
    });
}

```

Which then maybe feels like the correct spelling is actually more like this, so that we can actually properly infer all the information:

```

constexpr auto property(string_view name, meta::info type)
    -> void
{
    auto member = inject(data_member_spec{
        .name=std::format("m_{}", name),
        .type=type
    });

    // note that this type is structural
    struct Context {
        std::meta::info type;
        std::meta::info member;
    };
    auto pctx = Context{
        // get the type of the current context that we're injecting into
        .type=type_of(std::meta::current()),
        .member=member
    };

    inject(function_member_spec{
        .name=std::format("get_{}", name),
        .body=defer(context, ^[]<Context C>(){
            return [] (typename [:C.type:] const& self) -> auto const& {

```

```

        return self.[:C.member:];
    };
}
});

inject(function_member_spec{
    .name=std::format("set_{}", name),
    .body=defer(context, ^[]<Context C>(){
        return [] (typename [:C.type:]& self, typename
            [:type_of(C.member):] const& x) -> void {
            self.[:C.member:] = x;
        };
    })
});
}

```

That is, we create a custom context type that we pass in as a non-type template parameter into a lambda, so that it *it* can return a new lambda with all the types and names properly substituted in when that can actually be made to work.

This solution... might be workable. But it's already pretty complicated and the problem we're trying to solve really isn't. As a result, we believe that the spec API is somewhat of a dead end when it comes to extending injection support.

§ 2.1.4 Disposition

It's hard to view favorably a design for the long-term future of code injection with which we cannot even figure out how to inject functions. Even if we could, this design scales poorly with the language: we need a library for API for many language constructs, and C++ is a language with a lot of kinds. That makes for a large barrier to entry for metaprogramming that we would like to avoid.

Nevertheless, the spec API does have one thing going for it: it is quite simple. At the very least, we think we should extend the spec model in P2996 in the following ways:

- extend `data_member_spec` to support all data members (static/constexpr/inline, attributes, access, and initializer).
- add `alias_spec` and `base_class_spec`

These are the simple cases, and we can get a lot done with the simple cases, even without a full solution.

§ 2.2 The CodeReckons API

The [CodeReckons](#) approach provides a very different injection mechanism than what is in P2996 or what has been described in any of the metaprogramming papers. We can run through these three examples and see what they look like. Here, we will use the actual syntax as implemented in that compiler.

§ 2.2.1 `std::tuple`

The initial CodeReckons article provides [an implementation](#) for adding the data members of a tuple like so:

```
template <class... Ts>
struct tuple {
    % [] (class_builder& b) {
        int k = 0;
        for (type T : std::meta::type_list{^Ts...}) {
            append_field(b, cat("m", k++), T);
        }
    }();
};
```

This isn't too different from what we showed in the earlier section with `data_member_spec`. Different spelling and API, but it's the same model (`append_field` is equivalent to injecting a `data_member_spec`).

§ 2.2.2 `std::enable_if`

Likewise, we have just a [difference of spelling](#):

```
template <bool B, typename T=void>
struct enable_if {
    % [] (class_builder& b) {
        if (B) {
            append_alias(b, identifier{"type"}, ^T);
        }
    }();
};
```

§ 2.2.3 Properties

Here is where the CodeReckons approach differs greatly from the potential spec API, and it's worth examining how they [got it working](#):

```
constexpr auto property(class_builder& b, type type, std::string name) -> void {
    auto member_name = identifier{"m_" + name}.c_str();
    append_field(b, member_name, type);

    // getter
    {
        method_prototype mp;
        object_type(mp, make_const(decl_of(b)));
        return_type(mp, make_lvalue_reference(make_const(type)));

        append_method(b, identifier{"get_" + name}.c_str(), mp,
            [member_name] (method_builder& b) {
                append_return(b,
                    make_field_expr(
```

```

        make_deref_expr(make_this_expr(b)),
        member_name));
    });
}

// setter
{
    method_prototype mp;
    append_parameter(mp, "x",
        make_lvalue_reference(make_const(type)));
    object_type(mp, decl_of(b));
    return_type(mp, ^void);

    append_method(b, identifier{"set_" + name}.c_str(), mp,
        [member_name](method_builder& b){
            append_expr(b,
                make_operator_expr(
                    operator_kind::assign,

            make_field_expr(make_deref_expr(make_this_expr(b)),
                member_name),
                make_decl_ref_expr(parameters(decl_of(b))[1])
            ));
        });
}

struct Book {
    % property(^std::string, "author");
    % property(^std::string, "title");
};

```

In this model, we have to provide the signature of the two member functions (via `method_prototype`), and the bodies of the two member functions are provided as lambdas. But the lambda bodies here are not the C++ code that will be evaluated at runtime - it's still part of the AST building process. We have to define, at the AST level, what these member functions do.

In the spec API, we struggled how to write a function that takes a string `const&` and whose body is `self.{member name} = x;`. Here, because we don't need to access any of our reflections as constant expressions, we can make use of them directly.

But the result is... extremely verbose. This is a lot of code, that doesn't seem like it would scale very well. The setter alone (which is just trying to do something like `self.m_author = x;`) is already 14 lines of code and is fairly complicated. We think it's important that code injection still look like writing C++ code, not live at the AST level.

Nevertheless, this API does actually work. Whereas the spec API is still, at best, just a guess.

§ 2.2.4 Postfix Increment

For postfix increment, we want to inject the single function:

```

auto operator++(int) -> T {
    auto tmp = *this;
    ++*this;
    return tmp;
}

```

We rely on the type to provide the correct prefix increment. With the CodeReckons API, that looks [like this](#):

```

consteval auto postfix_increment(class_builder& b) -> void {
    method_prototype mp;
    append_parameter(mp, "x", ^int);
    object_type(mp, decl_of(b));
    return_type(mp, decl_of(b));

    append_method(b, operator_kind::post_inc, mp,
        [](method_builder& b){
            // auto tmp = *this;
            auto tmp = append_var(b, "tmp", auto_ty,
                make_deref_expr(make_this_expr(b)));
            // ++*this;
            append_expr(b,
                make_operator_expr(
                    operator_kind::pre_inc,
                    make_deref_expr(make_this_expr(b))));
            // return tmp;
            append_return(b, make_decl_ref_expr(tmp));
        });
}

struct C {
    int i;

    auto operator++() -> C& {
        ++i;
        return *this;
    }

    % postfix_increment();
};

```

As with the property example above, having an AST-based API is extremely verbose. It might be useful to simply compare the statement we want to generate with the code that we require to write to generate that statement:

```

// auto tmp = *this;
    auto tmp = append_var(b, "tmp", auto_ty,
        make_deref_expr(make_this_expr(b)));

// ++*this;

```

```

        append_expr(b, make_operator_expr(operator_kind::pre_inc,
        make_deref_expr(make_this_expr(b))));

// return tmp;
append_return(b, make_decl_ref_expr(tmp));

```

§ 2.2.5 Disposition

We believe an important goal for code injection is that the code being injected looks like C++. This is the best way to ensure both a low barrier to entry for using the facility as well as easing language evolution in the future. We do not want to have to add a mirror API to the reflection library for every language facility we add.

The CodeReckons API has the significant and not-to-be-minimized property that it, unlike the Spec API, works. It is also arguably easy to *read* the code in question to figure out what is going on. In our experiments with simply giving people code snippets to people with no context and asking them what the snippet does, people were able to figure it out.

However, in our experience it is pretty difficult to *write* the code precisely because it needs to be written at a different layer than C++ code usually is written in and the abstraction penalty (in terms of code length) is so large. We will compare this AST-based API to a few other ideas in the following sections to give a sense of what we mean here.

§ 2.3 String Injection

If we go back all the way to the beginning - we're trying to inject code. Perhaps the simplest possible model for how to inject code would be: just inject strings.

The advantage of strings is clear: everyone already knows how to build up strings. This makes implementing the three use-cases presented thus far is pretty straightforward.

§ 2.3.1 `std::tuple`

We could just do tuple this way:

```

template <class... Ts>
struct Tuple {
    constexpr {
        std::array types{^Ts...};
        for (size_t i = 0; i != types.size(); ++i) {
            inject(std::format(
                "[[no_unique_address]] {} _{};",
                qualified_name_of(types[i]),
                i));
        }
    };
};

```

Note that here we even added support for `[[no_unique_address]]`, which we haven't done in either of the previous models. Although we could come up with a way to add it to either of the two previous APIs, the fact that with string injection we don't even have to come up with a way to do this is a pretty significant upside. Everything just works.

Now, this would work - we'd have to be careful to use `qualified_name_of` here to avoid any question of name lookup. But it would be better to simply avoid these questions altogether by actually being able to splice in the type rather than referring to it by name.

We can do that by very slightly extending the API to take, as its first argument, an environment. And then we can reduce it again by having the API itself be a format API:

```
template <class... Ts>
struct Tuple {
    consteval {
        std::array types{^Ts...};
        for (size_t i = 0; i != types.size(); ++i) {
            inject(
                {"type", types[i]},
                "[[no_unique_address]] [:type:] _{};",
                i);
        }
    }
};
```

§ 2.3.2 `std::enable_if`

This one is even simpler, since we don't even need to bother with name lookup questions or splicing:

```
template <bool B, class T=void>
struct enable_if {
    consteval {
        if (B) {
            inject("using type = T;");
        }
    }
};
```

§ 2.3.3 Properties

Unlike with the spec API, implementing a property by way of code is straightforward. And unlike the CodeReckons API, we can write what looks like C++ code:

```
consteval auto property(info type, string_view name) -> void {
    inject(meta::format_with_environment(
        {"T", type}},
        R"(
private:
```

```

        [:T:] m_{0};

    public:
        auto get_{0}() const -> [:T:] const& {{
            return m_{0};
        }}

        auto set_{0}(typename [:T:] const& x) -> void {{
            m_{0} = x;
        }}
    }",
    name));
}

struct Book {
    consteval {
        property(^string, "author");
        property(^string, "title");
    }
}

```

§ 2.3.4 Postfix Increment

Similarly, the postfix increment implementation just writes itself. In this case, we can even return `auto` so don't even need to bother with how to spell the return type:

```

consteval auto postfix_increment() -> void {
    inject(R"(
        auto operator++(int) {
            auto tmp = *this;
            ++*this;
            return tmp;
        }
    )");
}

struct C {
    int i;

    auto operator++() -> C& {
        ++i;
        return *this;
    }

    consteval { postfix_increment(); }
};

```

§ 2.3.5 Disposition

Can pretty much guarantee that strings have the lowest possible barrier to entry of any code injection API. Which is a benefit that is not to be taken lightly! It is not surprising that D and Jai both have string-

based injection mechanisms.

But string injection is hardly perfect, and several of the issues with it might be clear already:

1. String injection does let you write what looks like C++ code, but it wouldn't let you use any macros - as those don't affect the contents of string literals and we can't run another preprocessing step later.
2. Our main string formatting mechanism, `format`, uses `{}` for replacement fields, which means actual braces - which show up in C++ a lot - have to be escaped. It also likely isn't the most compile-time efficient API, so driving reflection off of it might be suboptimal.
3. You don't get syntax highlighting for injected code strings. They're just strings. Perhaps we could introduce a new kind of string literal that syntax highlighters could recognize, but that seems like pre-emptively admitting defeat.
4. Errors happen at the point of *injection*, not at the point where you're writing the code. And the injection could happen very far away from the code.

But string injection offers an extremely significant advantage that's not to be underestimated: everyone can deal with strings and strings already just support everything, for all future evolution, without the need for a large API.

Can we do better?

§ 2.4 Fragments

[P1717R0] introduced the concept of fragments. It introduced many different kinds of fragments, under syntax that changed a bit in [P2050R0] and [P2237R0]. We'll use what the linked implementation uses, but feel free to change it as you read.

§ 2.4.1 `std::tuple`

The initial fragments paper itself led off with an implementation of `std::tuple` storage and the concept of a `constexpr` block (now also [P3289R0]). That [looks like this](#) (the linked implementation looks a little different, due to an implementation bug):

```
template<class... Ts>
struct Tuple {
    constexpr {
        std::array types{^Ts...};
        for (size_t i = 0; i != types.size(); ++i) {
            -> fragment struct {
                [[no_unique_address]] typename(%{types[i]})
                unqualid("_", %{i});
            };
        }
    };
};
```

Now, the big advantage of fragments is that it's just C++ code in the middle there (maybe it feels a bit messy in this example but it will more shortly). The leading `->` is the injection operator.

One big problem that fragments need to solve is how to get context information into them. For instance, how do we get the type `types[i]` and how do we produce the names `_0`, `_1`, ..., for all of these members? We need a way to capture context, and it needs to be interpolated differently.

In the above example, the design uses the operator `unqualid` (to create an unqualified id) concatenating the string literal `"_"` with the interpolated value `%{i}` (a later revision used `|# #|` instead). We need distinct operators to differentiate between the case where we want to use a string as an identifier and as an actual string.

```
std::string name = "hello";
-> fragment struct {
    // std::string name = "name";
    std::string unqualid(%{name}) = %{name};
};
```

§ 2.4.2 `std::enable_if`

It is very hard to compete [with this](#):

```
template <bool B, class T=void>
struct enable_if {
    consteval {
        if (B) {
            -> fragment struct { using type = T; };
        }
    };
};
```

Sure, you might want to simplify this just having a class scope `if` directly and then putting the contents of the fragment in there. But this is very nice.

§ 2.4.3 Properties

The [implementation here](#) isn't too different from the [string implementation](#) (this was back when the reflection operator was `reflexpr`, before it changed to `^`):

```
consteval auto property(meta::info type, char const* name) -> void {
    -> fragment struct {
        typename(%{type}) unqualid("m_", %{name});

        auto unqualid("get_", %{name})() -> typename(%{type}) const& {
            return unqualid("m_", %{name});
        }
    };
}
```



```

        auto unqualid("set_", %{name})(typename(%{type}) const& x) ->
        void {
            unqualid("m_", %{name}) = x;
        }
    };

    struct Book {
        consteval {
            property(reflexpr(std::string), "author");
            property(reflexpr(std::string), "title");
        }
    };

```

It's a bit busy because nearly everything in properties involves quoting outside context, so seemingly everything here is quoted.

Now, there's one very important property of fragments (as designed in these papers) hold: every fragment must be parsable in its context. A fragment does not leak its declarations out of its context; only out of the context where it is injected. Not only that, we get full name lookup and everything.

On the one hand, this seems like a big advantage: the fragment is checked at the point of its declaration, not at the point of its use. With the string model above, that was not the case - you can write whatever garbage string you want and it's still a perfectly valid string, it only becomes invalid C++ code when it's injected.

On the other, it has some consequences for how we can code using fragments. In the above implementation, we inject the whole property in one go. But let's say we wanted to split it up for whatever reason. We can't. This is invalid:

```

consteval auto property(meta::info type, char const* name) -> void
{
    -> fragment struct {
        typename(%{type}) unqualid("m_", %{name});
    };

    -> fragment struct {
        auto unqualid("get_", %{name})() -> typename(%{type}) const& {
            return unqualid("m_", %{name}); // error
        }

        auto unqualid("set_", %{name})(typename(%{type}) const& x) ->
        void {
            unqualid("m_", %{name}) = x; // error
        }
    };
}

```

In this second fragment, name lookup for `m_author` fails in both function bodies. We can't do that. We have to teach the fragment how to find the name, which requires writing this (note the added

requires statement):

```
constexpr auto property(meta::info type, char const* name) -> void
{
    -> fragment struct {
        typename(%{type}) unqualid("m_", %{name});
    };

    -> fragment struct {
        requires typename(%{type}) unqualid("m_", %{name});

        auto unqualid("get_", %{name})() -> typename(%{type}) const& {
            return unqualid("m_", %{name}); // error
        }

        auto unqualid("set_", %{name})(typename(%{type}) const& x) ->
        void {
            unqualid("m_", %{name}) = x; // error
        }
    };
}
```

§ 2.4.4 Postfix increment

One boilerplate annoyance is implementing `x++` in terms of `++x`. Can code injection help us out? Postfix increment ends up being [much simpler to implement](#) with fragments than properties - due to not having to deal with any quoted names. But it does surface the issue of name lookup in fragments.

```
constexpr auto postfix_increment() {
    -> fragment struct T {
        requires T& operator++();

        auto operator++(int) -> T {
            auto tmp = *this;
            ++*this;
            return tmp;
        }
    };
}

struct C {
    int i;

    auto operator++() -> C& {
        ++i;
        return *this;
    }

    constexpr { postfix_increment(); }
};
```

Now, the rule in the fragments implementation is that the fragments themselves are checked. This includes name lookup. So any name used in the body of the fragment has to be found and pre-declared, which is what we're doing in the `requires` clause there. The implementation right now appears to have a bug with respect to operators (if you change the body to calling `inc(*this)`, it does get flagged), which is why it's commented out in the link.

§ 2.4.5 Disposition

The fragment model seems substantially easier to program in than the CodeReckons model. We're actually writing C++ code. Consider the difference here between the CodeReckons solution and the Fragments solution to postfix increment:

CodeReckons	Fragments
<pre> consteval auto postfix_increment(class_builder& b) -> void { method_prototype mp; append_parameter(mp, "x", ^int); object_type(mp, decl_of(b)); return_type(mp, decl_of(b)); append_method(b, operator_kind::post_inc, mp, [](method_builder& b){ auto tmp = append_var(b, "tmp", auto_ty, make_deref_expr(make_this_expr(b))); append_expr(b, make_operator_expr(operator_kind::pre_inc, make_deref_expr(make_this_expr(b)))); append_return(b, make_decl_ref_expr(tmp)); }); } struct C { int i; auto operator++() -> C& { ++i; return *this; } % postfix_increment(); }; </pre>	<pre> consteval auto postfix_increment() { -> fragment struct T { requires T& operator++ (); auto operator++(int) - > T { auto tmp = *this; ++*this; return tmp; } }; struct C { int i; auto operator++() -> C& { ++i; return *this; } consteval { postfix_increment(); } }; </pre>

We lined up the fragment implementation to roughly correspond to the CodeReckons API on the left. With the code written out like this, it's easy to understand the CodeReckons API. But it takes no time at all to understand (or write) the fragments code on the right - it's just C++ already.

We also think it's a better idea than the string injection model, since we want something with structure that isn't just missing some parts of the language (the processor) and plays nicely with tools (like syntax highlighters).

But we think the fragment model still isn't quite right. By nobly trying to diagnose errors at the point of fragment declaration, it adds a complexity to the fragment model in a way that we don't think carries its

weight. The fragment papers ([[P1717R0](#)] and [[P2237R0](#)]) each go into some detail of different approaches of how to do name checking at the point of fragment declaration. They are all complicated.

We basically want something between strings and fragments.

§ 3 Token Sequences

Generation of code from low-level syntactic elements such as strings or token sequences may be considered quite unsophisticated. Indeed, previous proposals for code synthesis in C++ have studiously minimized using strings or tokens as input but resorting to AST-based APIs, expansion statements, or code fragments, as shown above. As noted by Andrew Sutton in [[P2237R0](#)]:

synthesizing new code from strings is straightforward, especially when the language/library has robust tools for compile-time string manipulation [...] the strings or tokens are syntactically and semantically unanalyzed until they are injected

whereas the central premise—and purported advantage—of a code fragment is it

should be fully syntactically and semantically validated prior to its injection

Due to the lack of consensus for a code synthesis mechanism, some C++ reflection proposals shifted focus to the query side of reflection and left room for scant code synthesis capabilities.

After extensive study and experimentation (as seen above), we concluded that some crucially important forms of token synthesis are necessary for practical code generation, and that insisting upon early syntactic and semantic validation of generated code is a net liability. The very nature of code synthesis involves assembling meaningful constructs out of pieces that have little or no meaning in separation. Using concatenation and deferring syntax/semantics analysis to offer said concatenation is by far the simplest, most direct approach to code synthesis.

Generally, we think that imposing early checking on generated code is likely to complicate and restrict the ways in which users can use the facility and also be difficult for implementers, thus hurting everyone involved.

We therefore acknowledge the notion of token sequence as a core building block for generating code. Using token sequences allows flexibility to code that generates other code, while deferring name lookup and semantic analysis to well-defined points in the compilation process. Thus we reach the notion of a `@tokens` literal dedicated to representing unprocessed sequences of tokens.

§ 3.1 `@tokens` literal

We propose the introduction of a new literal with the following syntax (the specific introducer can be decided later):

```
@tokens { balanced-brace-tokens }
```

where *balanced-brace-tokens* is an arbitrary sequence of C++ tokens with the sole requirement that the `{` and `}` pairs are balanced. Parentheses and square brackets may be unbalanced. The opening and closing `{/}` are not part of the token sequence. The type of a `@tokens` literal is `std::meta::info`.

For example:

```
constexpr auto t1 = @tokens { a + b };           // three tokens  
static_assert(std::is_same_v<decltype(t1), const std::meta::info>);  
constexpr auto t2 = @tokens { a += ( );         // code does not have to be  
                                meaningful  
constexpr auto t3 = @tokens { abc { def };      // Error, unpaired brace
```

Token sequences can be concatenated with the `+` operator. The result is a token sequence consisting of the concatenation of the operands.

```
constexpr auto t1 = @tokens { c = };  
constexpr auto t2 = @tokens { a + b; };  
constexpr auto t3 = t1 + t2;  
static_assert(t3 == @tokens { c = a + b; });
```

The concatenation is not textual - two tokens concatenated together preserve their identity, they are not pasted together into a single token. For example:

```
constexpr auto t1 = @tokens { abc };  
constexpr auto t2 = @tokens { def };  
constexpr auto t3 = t1 + t2;  
static_assert(t3 != @tokens { abcdef });  
static_assert(t3 == @tokens { abc def });
```

Whitespace and comments are treated just like in regular code - they are not significant beyond their role as token separator. For example:

```
constexpr auto t1 = @tokens { hello = /* world */ "world" };  
constexpr auto t2 = @tokens { /* again */ hello="world" };  
static_assert(t1 == t2);
```

Because tokens are handled after the the initial phase of preprocessing, macros and string concatenation can apply - but you have to be careful with macros because they won't work the way you might want

```
static_assert(tokens { "abc" "def" } ==tokens { "abcdef" });
```

```

// this concatenation produces the token sequence "abc" "def", not
// "abcdef"
// when this token sequence will be injected, that will be ill-formed
static_assert(tokens { "abc" } + tokens { "def" } != @tokens { "abcdef"
    });

#define PLUS_ONE(x) ((x) + 1)
static_assert(tokens { PLUS_ONE(x) } == tokens { ((x) + 1) });

// amusingly this version also still works but not for the reason you
// think
// on the left-hand-side the macro PLUS_ONE is still invoked...
// but as PLUS_ONE(x) +@tokens{
// which produces ((x) +@tokens{) + 1)
// which leads to tokens { ((x) +@tokens{) + 1) }
// which is @tokens{ ((x) + 1)}
static_assert(tokens { PLUS_ONE(x) } + tokens{ ) } == @tokens {
    PLUS_ONE(x) });

// But this one finally fails, because the macro isn't actually invoked
constexpr auto tok2 = []{
    auto t = @tokens { PLUS_ONE(x );
    constexpr_print_str("Logging...\n");
    t += @tokens{ ) }
    return t;
}();
static_assert(tok2 != @tokens { PLUS_ONE(x) });

```

A token sequence has no meaning by itself, until injected. But because (hopefully) users will write valid C++ code, the resulting injection actually does look like C++ code.

§ 3.2 Quoting into a token sequence

There's still the issue that you need to access outside context from within a token sequence. For that we introduce dedicated capture syntax using the interpolators `$eval` and `$id`.

The implementation model for this is that we collect the tokens within a `@token { ... }` literal, but every time we run into a capture, we parse and evaluate the expression within and replace it with the value as described below:

- `$eval(e)` for `e` of type `meta::info` is replaced by a pseudo-literal token holding the `info` value. If `e` is itself a token sequence, the contents of that token sequence are concatenated in place.
- `$id(e)` for `e` being string-like or integral is replaced with that value. `$id(e...)` can concatenate multiple string-like or integral values into a single identifier.

These need to be distinct because a given string could be intended to be injected as a *string*, like `"var"`, or as an *identifier*, like `var`. There's no way to determine which one is intended, so they have to be spelled differently.

With that in mind, we can start going through our examples.

§ 3.3 Examples

Now, the `std::tuple` and `std::enable_if` cases would look identical to their corresponding implementations with [fragments](#). In both cases, we are injecting complete code fragments that require no other name lookup, so there is not really any difference between a token sequence and a proper fragment. You can see the use of both kinds of interpolator on the left:

<code>std::tuple</code>	<code>std::enable_if</code>
<pre>template<class... Ts> struct Tuple { consteval { std::array types{^Ts...}; for (size_t i = 0; i != types.size(); ++i) { inject(@tokens { [[no_unique_address]] [: \$eval(types[i]) :] \$id("_", i); }); } } };</pre>	<pre>template <bool B, class T=void> struct enable_if { consteval { if (B) { inject(@tokens { using type = T; }); } }; };</pre>

The property example likewise could be identical, but we do not run into any name lookup issues, so we can write it any way we want - either as injecting one token sequence or even injecting three. Both work fine without needing any additional declarations:

Single Token Sequence	Three Token Sequences
<pre>consteval auto property(meta::info type, std::string name) -> void { std::string member_name = "m_" + name; inject(@tokens { [:\$eval(type):] \$id(member_name); auto \$id("get_", name)() -> [:\$eval(type):] const& { return \$id(member_name); } auto \$id("set_", name)(typename [:\$eval(type):] const& x) -> void { \$id(member_name) = x; } }); }</pre>	<pre>consteval auto property(meta::info type, std::string name) -> void { std::string member_name = "m_" + name; inject(@tokens { [:\$eval(type):] \$id(member_name); }); inject(@tokens { auto \$id("get_", name)() -> [:\$eval(type):] const& { return \$id(member_name); } }); inject(@tokens { auto \$id("set_", name)(typename [:\$eval(type):] const& x) }); }</pre>

Single Token Sequence	Three Token Sequences
<pre>}); }</pre>	<pre>-> void { \$id(member_name) = x; } }); }</pre>

With the postfix increment example, we see some more interesting difference. We are not proposing any special-case syntax for getting at the type that we are injecting into, so it would have to be pulled out from the context (we'll name it T in both places for consistency):

Fragment	Token Sequence
<pre>constexpr auto postfix_increment() -> void { -> fragment struct T { requires T& operator++(); auto operator++(int) -> T { auto tmp = *this; ++*this; return tmp; } }; }</pre>	<pre>constexpr auto postfix_increment() -> void { auto T = type_of(std::meta::current()); inject(@tokens { auto operator++(int) -> [:\$eval(T):] { auto tmp = *this; ++*this; return tmp; } }); }</pre>

The syntax here is, unsurprisingly, largely the same. We're mostly writing C++ code. The difference is that we no longer need to pre-declare the functions we're using and the feature set is smaller. While declaring T as part of the fragment is certainly convenient, we're shooting for a smaller feature.

§ 3.4 Logging Vector: Cloning a Type

The goal here is we want to implement a type `LoggingVector<T>` which behaves like `std::vector<T>` in all respects except that it prints the function being called.

We start with this:

```
template <typename T>
class LoggingVector {
    std::vector<T> impl;

public:
    LoggingVector(std::vector<T> v) : impl(std::move(v)) { }

    constexpr {
        for (std::meta::info fun : /* public, non-special member
            functions */) {
            inject(@tokens {
                declare [:$eval(decl_of(fun)) :] {
```

```

// ...
};
}
});
}
};

```

We want to clone every member function, which requires copying the declaration. We don't want to actually have to spell out the declaration in the token sequence that we inject - that would be a tremendous amount of work given the complexity of C++ declarations. So instead we introduce a new kind of splice: a declaration splice. We already have `typename [: e :]` and `template [: e :]` in other contexts, so `declare [: e :]` at least fits within the family of splicers.

Now, we have two problems to solve in the body (as well as a few more problems we'll get to later).

First, we need to print the name of the function we're calling. This is easy, since we have the function and can just ask for its name.

Second, we need to actually forward the parameters of the function into our member impl. This is, seemingly, very hard:

```

consteval {
  for (std::meta::info fun : /* public, non-special member functions
    */) {
    inject(@tokens {
      declare [: $eval(decl_of(fun)) :] {
        std::println("Calling {}", $eval(name_of(fun)));
        return impl.[: $eval(fun) :](/* ??? */);
      }
    });
  }
}

```

This is where the ability of token sequences to be concatenated from purely sequences of tokens really gives us a lot of value. How do we forward the parameters along? We don't even have the parameter names here - the declaration that we're cloning might not even *have* parameter names.

So there are two approaches that we can use here:

§ 3.4.1 Reflections of Parameters

We need the ability to just ask for the parameters themselves (which [\[P3096R0\]](#) should provide). And then the goal here is to inject the tokens for the call:

```

return impl.[: fun:]([: p0:], [: p1:], ..., [: pn:])

```

But the tricky part is that we can't ask for the parameters of the function we're cloning (i.e. `fun` in the loop above - which is a reflection of a non-static member function of `std::vector<T>`), we have to ask for the parameters of the function that we're *currently* defining. Which we haven't defined yet so we can't reflect on it.

But we could split this in pieces and ask `inject` to give us back a reflection of what it injected, since `inject` must operate on full token boundaries.

So that might be:

```
template <typename T>
class LoggingVector {
    std::vector<T> impl;

public:
    LoggingVector(std::vector<T> v) : impl(std::move(v)) { }

    consteval {
        for (std::meta::info fun : /* public, non-special member
            functions */) {
            // note that this one doesn't even require a token sequence
            auto log_fun = inject(decl_of(fun));

            auto argument_list = @tokens { };
            bool first = true;
            for (auto param : parameters_of(log_fun)) { // <== NB, not
                fun
                if (not first) {
                    argument_list += @tokens { , };
                }
                first = false;
                argument_list += @tokens {
                    static_cast<[:$eval(type_of(param)):]&&>([:
$eval(param) :])
                };
            }

            inject(@tokens {
                declare [: $eval(decl_of(fun)) :] {
                    std::println("Calling {}", $eval(name_of(fun)));
                    return impl.[: $eval(fun) :]( $eval(argument_list)
                );
            });
        }
    }
};
```

The `argument_list` is simply building up the token sequence `[: p0 :]`, `[: p1 :]`, ..., `[: pN :]` for each parameter (except forwarded). There is no name lookup going on, no checking of fragment correctness. Just building up the right tokens.

Once we have those tokens, we can concatenate this token sequence using the same `$eval()` quoting operator that we've used for other problems and we're done. Token sequences are just a sequence of tokens - so we simply need to be able to produce that sequence.

Note that we didn't actually have to implement it using a separate `argument_list` local variable - we could've concatenated the entire token sequence piecewise. But this structure allows factoring out parameter-forwarding into its own function:

```
constexpr auto forward_parameters(std::meta::info fun) ->
    std::meta::info {
    auto argument_list = @tokens { };
    bool first = true;
    for (auto param : parameters_of(fun)) {
        if (not first) {
            argument_list += @tokens { , };
        }
        first = false;
        argument_list += @tokens {
            static_cast<[:$eval(type_of(param)):]&&>([: $eval(param) :])
        };
    }
    return argument_list;
}
```

And then:

```
constexpr {
    for (std::meta::info fun : /* public, non-special member functions
    */) {
        auto log_fun = inject(decl_of(fun));

        inject(@tokens {
            declare [: $eval(decl_of(fun)) :] {
                std::println("Calling {}", $eval(name_of(fun)));
                return impl.[: $eval(fun) :](
                    $eval(forward_parameters(log_fun)) );
            }
        });
    }
}
```

§ 3.4.2 Introducing Parameter Names

We said we have the problem that the functions we're cloning might not have parameter names. So what? We're creating a new function, we can pick our names!

Perhaps that looks like an extra argument to `decl_of` that gives us a prefix for each parameter name. So `decl_of(fun, "p")` would give us parameter names of `p0`, `p1`, and so forth. That gives us a similar looking solution, but now we never need the reflection of the new function - just the old one:

```

template <typename T>
class LoggingVector {
    std::vector<T> impl;

public:
    LoggingVector(std::vector<T> v) : impl(std::move(v)) { }

    consteval {
        for (std::meta::info fun : /* public, non-special member
            functions */) {
            auto argument_list = @tokens { };
            for (size_t i = 0; i != parameters_of(fun).size(); ++i) {
                if (i > 0) {
                    argument_list += @tokens { , };
                }

                argument_list += @tokens {
                    // we could get the nth parameter's type (we can't
                    splice
                    // the other function's parameters but we CAN query
                    them)
                    // or we could just write decltype(p0)
                    static_cast<decltype($id("p", i))&&>($id("p", i))
                };
            }

            inject(@tokens {
                declare [: $eval(decl_of(fun, "p")) :] {
                    std::println("Calling {}", $eval(name_of(fun)));
                    return impl.[: $eval(fun) :]( $eval(argument_list)
                );
            });
        }
    }
};

```

This approach is arguably simpler.

§ 3.5 Logging Vector II: Cloning with Modifications

However, we've still got some work to do. The above implementation already gets us a great deal of functionality, and should create code that looks something like this:

```

template <typename T>
class LoggingVector {
    std::vector<T> impl;

public:
    LoggingVector(std::vector<T> v) : impl(std::move(v)) { }

    auto clear() -> void {

```

```

        std::println("Calling {}", "clear");
        return impl.clear();
    }

    auto push_back(T const& value) -> void {
        std::println("Calling {}", "push_back");
        return impl.push_back(static_cast<T const&>(value));
    }

    auto push_back(T&& value) -> void {
        std::println("Calling {}", "push_back");
        return impl.push_back(static_cast<T&&>(value));
    }

    // ...
};

```

For a lot of `std::vector`'s member functions, we're done. But some need some more work. One of the functions we're emitting is member `swap`:

```

template <typename T>
class LoggingVector {
    std::vector<T> impl;

public:
    // ...

    auto swap(std::vector<T>& other) noexcept(/* ... */) -> void {
        std::println("Calling {}", "swap");
        return impl.swap(other); // <== omitting the cast here for
                                   readability
    }

    // ...
};

```

But this... isn't right. Or rather, it could potentially be right in some design, but it's not what we want to do. We don't want `LoggingVector<int>` to be swappable with `std::vector<int>`... we want it to be swappable with itself. What we actually want to do is emit this:

```

    auto swap(LoggingVector<T>& other) noexcept(/* ... */) -> void {
        std::println("Calling {}", "swap");
        return impl.swap(other.impl);
    }

```

Two changes here: the parameter needs to change from `std::vector<T>&` to `LoggingVector<T>&`, and then in the call-forwarding we need to forward not `other` (which is now the wrong type) but rather `other.impl`. How can we do that? We don't quite have a good answer yet. But this is much farther than we've come with any other design.

§ 4 Hygienic Macros

C macros have a (well-deserved) bad reputation in the C++ community. This is because they have some intractable problems:

- C macros don't follow any scoping rules, and can change any code, anywhere. This is why they do not leak into or out of C++ modules.
- The C preprocessor is a language unto itself, that doesn't understand C++ syntax, with limited functionality that is very tedious to program in.

We think that C++ does need a code manipulation mechanism, and that token sequences can provide a much better solution than C macros.

§ 4.1 Forwarding

Consider the problem of forwarding. Forwarding an argument in C++, in the vast majority of uses, looks like `std::forward<T>(t)`, where `T` is actually the type `decltype(t)`. This is annoying to have to write, the operation is simply forwarding an argument but we need to provide two names anyway, and also has the downside of having to instantiate a function template (although compilers are moving towards making that a builtin).

Barry at some point proposed a specific language feature for this use-case ([[P0644R1](#)]). Later, there was a proposal for a hygienic macro system [[P1221R1](#)] in which forwarding would be implemented like this:

```
using fwd(using auto x) {  
    return static_cast<decltype(x)&&>(x);  
}  
  
auto old_f = [](auto&& x) { return std::forward<decltype(x)>(x); };  
auto new_f = [](auto&& x) { return fwd(x); };
```

With token sequences, we can achieve similar syntax:

```
constexpr auto fwd2(@tokens x) -> info {  
    return @tokens {  
        static_cast<decltype($eval(x))&&>($eval(x));  
    };  
}  
  
auto new_f2 = [](auto&& x) { return fwd2!(x); };
```

The logic here is that `fwd2!(x)` is syntactic sugar for `inject(fwd2(@tokens { x })).` We're taking a page out of Rust's book and suggesting that invoking a "macro" with an exclamation point does the injection. Seems nice to both have convenient syntax for token manipulation and a syntactic marker for it on the call-site.

We would have to figure out what we would want `fwd2!(std::pair<int, int>{1, 2})` to do. One of the issues of C macros is not understand C++ token syntax, so this argument would have to be parenthesized. But if we want to operate on the token level, this seems like a given.

Of course, `fwd2` is a regular C++ function. You have to invoke it through the usual C++ scoping rules, so it does not suffer that problem from C macros. And then the body is a regular C++ function too, so writing complex token manipulation is just a matter of writing complex C++ code - which is a lot easier than writing complex C preprocessor code.

§ 4.2 Assertion

Consider a different example (borrowed from [here](#)):

```
constexpr auto assert_eq(@tokens a,
                        @tokens b) -> info {
    return @tokens {
        do {
            auto sa = $eval(stringify(a));
            auto va = $eval(a);

            auto sb = $eval(stringify(b));
            auto vb = $eval(b);

            if (not (va == vb)) {
                std::println(
                    stderr,
                    "{} ({{}}) == {} ({{}}) failed at {}",
                    sa, va,
                    sb, vb,
                    $eval(source_location_of(a)));
                std::abort();
            }
        } while (false);
    };
}
```

With the expectation that:

Written Code	Injected Code
<pre>assert_eq!(42, factorial(3));</pre>	<pre>do { auto sa = "42"; auto va = 42; auto sb = "factorial(3)"; auto vb = factorial(3); if (not (va == vb)) { std::println(stderr, "{} ({{}}) == {} ({{}}) failed at {}",</pre>

Written Code	Injected Code
	<pre> sa, va, sb, vb, /* some source location */; std::abort(); } } while(false); </pre>

You can write this as a regular C macro today, but we bet it's a little nicer to read using this language facility.

Note that this still suffers from at least one C macro problem: naming. If instead of `assert_eq!(42, factorial(3))` we wrote `assert_eq!(42, sa * 2)`, then this would not compile - because name lookup in the `do-while` loop would end up finding the local variable `sa` declared by the macro. So care would have to be taken in all of these cases (otherwise we would have to come up with a way to introduce unique names).

§ 4.3 String Interpolation

Many programming languages support string interpolation. The ability to write something like `format!("x={x}")` instead of `format("x={}", x)`. It's a pretty significant feature when it comes to the ergonomics of formatting.

We can write it as a library:

```

// the actual parsing isn't interesting here.
// the goal is to take a string like "x={this->x:02} y={this->y:02}"
// and return {format_str="x={:02} y={:02}", args={"this->x", "this->y"}}
struct FormatParts {
    string_view format_str;
    vector<string_view> args;
};
constexpr auto parse_format_string(string_view) -> FormatParts;

constexpr auto format(string_view str) -> meta::info {
    auto parts = parse_format_string(str);

    auto tok = @tokens {
        // NB: there's no close paren yet
        // we're allowed to build up a partial fragment like this
        ::std::format($eval(parts.format_str)
    };

    for (string_view arg : parts.args) {
        tok += @tokens { , $eval(tokenize(arg)) };
    }

    tok += @tokens { ) };

```

```

    return tok;
}

```

In the previous example, we demonstrated the need for a way to convert a token sequence to a string. In this example, we need a way to convert a string to a token sequence. This doesn't involve parsing or any semantic analysis. It's *just* lexing.

Of course, this approach has limitations. We cannot fully faithfully parse the format string because at this layer we don't have types - we can't stop and look up what type `this->x` was, instantiate the appropriate `std::formatter<X>` and use it to tell us where the end of its formatter is. We can just count balanced `{}`s and hope for the best.

Similarly, something like `format!("{SOME_MACRO(x)}")` can't work since we're not going to rerun the preprocessor during tokenization. But I doubt anybody would even expect that to work.

But realistically, this would handily cover the 90%, if not the 99% case. Not to mention could easily adopt other nice features of string interpolation that show up in other languages (like Python's `f"{x =}"` which formats as `"x = 42"`) as library features. And, importantly, this isn't a language feature tied to `std::format`. It could easily be made into a library to be used by any logging framework.

§ 4.4 Control Flow Operator

A simpler example would be the control flow operator [P2561R2]. Many people already use a macro for this. A hygienic macro would be that much better:

```

consteval auto try_(@tokens expr) -> info {
    return @tokens {
        do {
            decltype(auto) _f = $eval(expr);

            using _R = [:return_type(std::meta::current_function()):];
            using _TraitsR = try_traits<_R>;

            try_traits<[:type_remove_cvref(type_of(^_f)):]>;
                                using _TraitsF =
                                _TraitsR;

            if (not _TraitsF::should_continue(_f)) {
                return
                _TraitsR::from_break(_TraitsF::extract_break(forward!(_f)));
            }

            do_return _TraitsF::extract_continue(forward!(_f));
        };
    };
}

```

This relies on `do` expressions [P2806R2] to give us something to inject.

§ 4.5 Alternate Syntax

We have two forms of injection in this paper:

- a metafunction `std::meta::inject` that takes an info (and maybe also returns an info), used through [token sequences](#).
- a trailing `!` used throughout [hygienic macros](#).

But these really are the exact same thing - both are requests to take an info and inject it in the current context. The bigger token sequence injection doesn't really have any particular reason to require terse syntax. Prior papers did use some punctuation marks (e.g. `->`, `<<`), but a named function seems better. But the macros *really* do want to have terse invocation syntax. Having to write `inject(forward(x))` somewhat defeats the purpose and nobody would write it.

Using one of the arrows for the macro use-case is weird, so one option might be prefix `@`. As in `@forward(x)`, `@assert_eq(a, b)`, and `@format("x={this->x}")`. This would mean that `@tokens { ... }` would need a different spelling, perhaps simply `^{ ... }`.

Or we could stick with two syntaxes - the longer one for the bigger reflection cases where terseness is arguably bad, and the short one for the macro use case where terseness is essential.

§ 5 Proposal

We propose a code injection mechanism using token sequences.

The fragment model initially introduced in [P1717R0] is great for allowing writing code-to-be-injected to actually look like regular C++ code, which has the benefit of being both familiar and being already recognizable to tools like syntax highlighters. But the early checking adds complexity to the model and the implementation which makes it harder to use and limits its usefulness. Hence, we propose raw token sequences that are unparsed until the point of injection.

This proposal consists of several pieces:

- a mechanism to introduce a token sequence (in this paper `@tokens { balanced-brace-tokens }`)
- two interpolators to add outside context to a token sequence, one for identifiers (`$id`) and one for values (`$eval`)
- a new disambiguator for splicing a declaration (`declare [ : fun : ]`)
- a new metafunction to inject a token sequence (`std::meta::inject`)
- new metaprogramming facilities for dealing with token sequences:
 - concatenation
 - converting a string to a token sequence and a token sequence to a string
 - splitting a token sequence into a range of tokens
- hygienic macros would benefit syntactically from:
 - a mechanism to accept a tokens sequence as a function parameter

— a mechanism to inject a token sequence directly as returned by a function (trailing !)

§ 6 References

- [P0644R1] Barry Revzin. 2017-10-08. Forward without forward.
<https://wg21.link/p0644r1>
- [P0707R4] Herb Sutter. 2019-06-17. Metaclasses: Generative C++.
<https://wg21.link/p0707r4>
- [P0712R0] Andrew Sutton, Herb Sutter. 2017-06-18. Implementing language support for compile-time programming.
<https://wg21.link/p0712r0>
- [P1221R1] Jason Rice. 2018-10-03. Parametric Expressions.
<https://wg21.link/p1221r1>
- [P1717R0] Andrew Sutton, Wyatt Childers. 2019-06-17. Compile-time Metaprogramming in C++.
<https://wg21.link/p1717r0>
- [P2050R0] Andrew Sutton, Wyatt Childers. 2020-01-13. Tweaks to the design of source code fragments.
<https://wg21.link/p2050r0>
- [P2237R0] Andrew Sutton. 2020-10-15. Metaprogramming.
<https://wg21.link/p2237r0>
- [P2561R2] Barry Revzin. 2023-05-18. A control flow operator.
<https://wg21.link/p2561r2>
- [P2806R2] Barry Revzin, Bruno Cardoso Lopez, Zach Laine, Michael Park. 2023-11-16. do expressions.
<https://wg21.link/p2806r2>
- [P2996R3] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, and Dan Katz. 2024-05-16. Reflection for C++26.
<https://wg21.link/p2996r3>
- [P3096R0] Adam Lach, Walter Genovese. 2024-02-14. Function Parameter Reflection in Reflection for C++26.
<https://wg21.link/p3096r0>
- [P3289R0] Wyatt Childers, Barry Revzin, and Daveed Vandevoorde. 2024-05-18. `consteval` blocks.
<https://wg21.link/p3289r0>